

LIFE733 202425 Assignment 2 File handling, functions, loops and dictionaries

Andy Jones, andrew.jones@liverpool.ac.uk

Deadline: 2pm on 12th November

Submission

- Upload to Canvas a single zip file containing:
 - o your python script(s) for part 1 and part 2,
 - o the output files from part 1a, 1b and part 2 as text files
 - o A word document, with screen shots showing your testing procedures
 - o Please remember to add some comments and descriptions to your code, especially your functions

Guidelines for Using GAI in .py Scripts and external libraries

Make sure to follow guidelines on Canvas, and declare any GAI usage. See also notes below about avoid the use of external libraries (except re library for regular expressions). The purpose of this assignment is to test your understanding of material in weeks 1-5 – pretty much everything you need is there: basic variables, conditionals, lists, loops, dictionaries, functions, files and regular expressions. This assignment will help you develop confidence putting them all together, to solve logical problems. As general advice, using a GAI tool as a “shortcut” means you will miss the key learning objectives.

Assignment overview

In this assignment, you will write some functions that can read in the DNA sequence of one chromosome from a fasta file¹, and then process a GFF3-formatted file² (Figure 1), which is a tab-separated file containing the coordinates of features on that chromosome (or genome).

You will read the coordinates (start and end positions) of each coding sequence (called “CDS” elements in GFF), and extract the corresponding sequences from the chromosome, using slicing. Most genes have more than one CDS (from different exons), so you need to use a dictionary to keep track of the gene identifier and its growing cDNA sequence. Once you have built up the cDNA sequence for each gene, you will write them out to a new output file. Next, you will use another dictionary, to look up the corresponding amino acid for each codon (3 bases), and convert the cDNA sequence to a protein sequence, and write the results to a second output file. Last, you will use regular expressions to find protein sequences containing a particular motif. Detailed instructions are given below.

¹ https://www.bioinformatics.nl/tools/crab_fasta.html

² <http://www.ensembl.org/info/website/upload/gff3.html>

Chromosome number	Feature type	Start	End	Strand	Transcript ID (following "ID=" up to ":" e.g. LOC_Os1g01010.1)
1	##gff-version 3				
2	Chr1 MSU_osalr7 gene	2903	10817	.	ID=LOC_Os01g01010;Name=LOC_Os01g01010;Note=TBC%20domain%20co
3	Chr1 MSU_osalr7 mRNA	2903	10817	.	ID=LOC_Os01g01010.1;Name=LOC_Os01g01010.1;Parent=LOC_Os01g01
4	Chr1 MSU_osalr7 exon	2903	3268	.	ID=LOC_Os01g01010.1:exon_1;Parent=LOC_Os01g01010.1
5	Chr1 MSU_osalr7 exon	3354	3616	.	ID=LOC_Os01g01010.1:exon_2;Parent=LOC_Os01g01010.1
6	Chr1 MSU_osalr7 exon	4357	4455	.	ID=LOC_Os01g01010.1:exon_3;Parent=LOC_Os01g01010.1
7	Chr1 MSU_osalr7 exon	5457	5560	.	ID=LOC_Os01g01010.1:exon_4;Parent=LOC_Os01g01010.1
8	Chr1 MSU_osalr7 exon	7136	7944	.	ID=LOC_Os01g01010.1:exon_5;Parent=LOC_Os01g01010.1
9	Chr1 MSU_osalr7 exon	8028	8150	.	ID=LOC_Os01g01010.1:exon_6;Parent=LOC_Os01g01010.1
10	Chr1 MSU_osalr7 exon	8232	8320	.	ID=LOC_Os01g01010.1:exon_7;Parent=LOC_Os01g01010.1
11	Chr1 MSU_osalr7 exon	8408	8608	.	ID=LOC_Os01g01010.1:exon_8;Parent=LOC_Os01g01010.1
12	Chr1 MSU_osalr7 exon	9210	9617	.	ID=LOC_Os01g01010.1:exon_9;Parent=LOC_Os01g01010.1
13	Chr1 MSU_osalr7 exon	10104	10187	.	ID=LOC_Os01g01010.1:exon_10;Parent=LOC_Os01g01010.1
14	Chr1 MSU_osalr7 exon	10274	10430	.	ID=LOC_Os01g01010.1:exon_11;Parent=LOC_Os01g01010.1
15	Chr1 MSU_osalr7 exon	10504	10817	.	ID=LOC_Os01g01010.1:exon_12;Parent=LOC_Os01g01010.1
16	Chr1 MSU_osalr7 five_prime_UTR	2903	3268	.	ID=LOC_Os01g01010.1:utr_1;Parent=LOC_Os01g01010.1
17	Chr1 MSU_osalr7 five_prime_UTR	3254	3448	.	ID=LOC_Os01g01010.1:utr_2;Parent=LOC_Os01g01010.1
18	Chr1 MSU_osalr7 CDS	3449	3616	.	ID=LOC_Os01g01010.1:cds_1;Parent=LOC_Os01g01010.1
19	Chr1 MSU_osalr7 CDS	4357	4455	.	ID=LOC_Os01g01010.1:cds_2;Parent=LOC_Os01g01010.1
20	Chr1 MSU_osalr7 CDS	5457	5560	.	ID=LOC_Os01g01010.1:cds_3;Parent=LOC_Os01g01010.1
21	Chr1 MSU_osalr7 CDS	7136	7944	.	ID=LOC_Os01g01010.1:cds_4;Parent=LOC_Os01g01010.1
22	Chr1 MSU_osalr7 CDS	8028	8150	.	ID=LOC_Os01g01010.1:cds_5;Parent=LOC_Os01g01010.1
23	Chr1 MSU_osalr7 CDS	8232	8320	.	ID=LOC_Os01g01010.1:cds_6;Parent=LOC_Os01g01010.1
24	Chr1 MSU_osalr7 CDS	8408	8608	.	ID=LOC_Os01g01010.1:cds_7;Parent=LOC_Os01g01010.1
25	Chr1 MSU_osalr7 CDS	9210	9617	.	ID=LOC_Os01g01010.1:cds_8;Parent=LOC_Os01g01010.1
26	Chr1 MSU_osalr7 CDS	10104	10187	.	ID=LOC_Os01g01010.1:cds_9;Parent=LOC_Os01g01010.1
27	Chr1 MSU_osalr7 CDS	10274	10297	.	ID=LOC_Os01g01010.1:cds_10;Parent=LOC_Os01g01010.1

Figure 1 The GFF file your code will read in, with the columns we are interested in flagged. Your first task is to extract the sequences with these coordinates from the chromosomal DNA sequence, and put them into a dictionary, using the gene ID as the dictionary key, and the concatenated sequence for all CDS elements for this gene as the value.

Important note: Later in the course, you will learn how you can do these tasks very simply using biopython. There are also lots of examples of the web of how to solve similar tasks. However, for this assignment, we want to see clear evidence that you have coded these tasks from first principles without using any modules or libraries (core python only). We will also check for high similarity to any web-based “model answers” for these types of things, so please do code it yourself! If you’re stuck, please ask for help.

Assignment Part 1a – Produce transcript sequences (50%)

Write a function that reads a FASTA formatted file³, **containing only a single DNA sequence**:

- The function should take as an input parameter the filename to read from
- Your function must remember to remove the end of line characters on each line
- Your function should remove “>” from the first line, and store this value as the **record_description**
- You should then concatenate each line onto a variable, e.g. called dna_seq
- Your function should return a list of two elements:
 - o Element 1: **The record_description**
 - o Element 2: **The DNA sequence** (containing no gaps or end of line characters)

Now use your function to read in the chromosome 1 DNA sequence.

For testing, just read in the first 1000 lines of data. Only once everything is working fine, you can run it once for the whole sequence.

Next, your code should create cDNA sequences from the chromosome sequence, by extracting blocks

with the coordinates found in the gff file. We are only interested in “CDS” rows i.e. coding parts of exons. You should read in the gff file using basic file reading, ignoring the header row, and splitting each row on tab characters (week 5 lecture materials). You may put this part in a function but you do not have to. *We do not want to see ChatGPT-inspired code that tries to use more advanced libraries like biopython or pandas for this task, and you will not be given any credit if you do not use basic file reading.*

For testing, only read in CDS elements with an end position < 60,000. Only once everything is working fine, you can run it once for the whole file.

Notes to help you:

- For each CDS element, use slicing to extract the relevant cDNA (coding DNA) chunk from the chromosome, noting that the coordinates in GFF start from 1, whereas the in python we start counting from zero *i.e.* `seq[start-1:end]`
- Build up the sequence for each transcript, using a dictionary here. Store `current_transcript` (see below) as the value slot of a dictionary, with the transcript identifier e.g. LOC_Os01g01010.1 as the dictionary ID. The transcript identifier comes from the final column, removing “ID=” and taking the text up to the first “.” character.
- As you read each line of the GFF file, check if you already put a cDNA sequence onto the dictionary for this ID. If not, start a new dictionary entry for this combination of ID -> transcript sequence. If yes, add the current sequence onto the existing sequence, and put the transcript sequence back into the value slot of the dictionary (see week 5 lecture materials, which explains this handy algorithm).
- For transcripts on the reverse strand (Figure 1, “-”), you should convert them to the reverse complement as you read them, before adding them onto the end of the sequence stored in the dictionary for this ID

Once you have read in all the transcripts, loop through all entries in the dictionary, use basic file writing (**not biopython**) for writing the transcripts back into a new text file in fasta format (example shown in Figure 2).



```

1 >LOC_Os01g01010.1
2 ATGTCGTCGCGCGCGGCCAAGACAACGAGATACCGCTGGGGACTACATCAAGTGGATGTGCGCGCGCGGTGGCCGTGCGGCGCGCGCCATGGCCAAGCTCCAGCGCGCGCGTTGGCTCCCTCGTCCGT
GACATTGGCGACCCCTGCCTCAACCCATCCCGGTTAAGGGGAGCAAAATGCTCAAACCGGAAAAATGGCACACATGTTTGTGATAATGATGGAAGGTCATAGGTTTCGTAAGGCCCTAAAATTCATT
GTCTTAGGGGGGTGGATCCCACTATTTCAGCTGAAGTTTGGGAATTTCTTCTGGCTGCTATGCCTTGAGTAGTACCTCAGAGTATAGGAGGAACTAAGAGCTGTGTAGAAGGGAAAAATATCAAAAT
TTAGTTAGACAGTGCCAGAGCATGCACCAAGCATTGGTACAGGTGAGCTTGCTTACGCTGTTGGATCAAAGCTAATGGATGTCAGGACTATGTCAAAAGAAACCCACATTGCAGAAAGATTAGCACA
AGTCAACAACTTCACAAATACAGCTGGCAGTTTAGTAGAAGATTGAGATTATGGCCCGGTTGGTGCACAGCAGTCACAAAAAGGGAAGCTGTAGTAAATCAGCTGAAGTGTGTTTCAACGTA
CATAATGATACCTCTCTTTATGATTCTCCAATTTTATTGTGTCTCTACAGAAGTGAATAATTGCTCAAAAGATTCTCAGGATTACAATGACATGGGTGAACCAAGATATGACACTGAGACCTTCGAT

```

Figure 2 The expected output from part 1a, partially shown for one sequence only.

Assignment Part 1b (30%)

Now you are going to turn those transcripts into proteins. You have two choices here. You can either start a new python file to read in the transcript file you created (i.e. make code `assignment2_1a.py` and `assignment2_1b.py`), or just work with the dictionaries you made in part 1a and add on to the same code i.e. single submission of `assignment2_1.py`.

Now do the following tasks:

- Create a **dictionary** e.g. called `codon_table`, where the keys are the triplets of DNA bases (codons) in upper case, and the values are amino acids (also upper case) in single letter code.

- Use the “Standard” table⁴
- **Write a function**, called `translate_dna` that:
 - Takes as an input parameter a DNA sequence (in upper or lower case – the latter handled appropriately).
 - In the function, loop through the sequence in increments of 3 (i.e. codons) starting from the first position (i.e. frame 1), doing look-ups in the dictionary to retrieve each amino acid for each codon.
 - Stop codons should be encoded as “*” for the corresponding look-up
 - Write additional code to check that a given codon is in the table, and print out an error message, if it is not.
 - Concatenate each amino acid to a variable called `prot_seq`, that gets returned at the end
 - Make sure to test your function first using some invented DNA sequences i.e. any combinations of A, C, G and T (*in your word document, show evidence of testing*)
- Test that you can create a protein sequence from the DNA sequence, and print it out to the terminal.

Final output, write out a fasta formatted file as shown in Figure 3, showing the protein sequences on rice chromosome 1. As a sanity check, each protein should end with one stop codon, and should have no stop codons in the middle. As above, for testing start with a small set before moving onto the full set, once you are happy with your code. You can also test if your protein sequences match the real set, downloadable from here:

https://rice.uga.edu/pub/data/Eukaryotic_Projects/o_sativa/annotation_dbs/pseudomolecules/version_7.0/all.dir/all.pep (contains proteins for all chromosomes, we have only chromosome 1 in this assignment).

```

1 >LOC_Os01g01010.1
2 MSSAAGQDNGDTAGDYIKWMCAGGRAGGAMANLQRGVGSILVRDIDGDCPLNPSVKGSKMLKPEKWHTCFDNDGKIVIGFRKALKFIVLGGVD
3 PTIRAEVWEFLGCGYALSSTSEYRRKLRAVRREKYQILVRQCQSMHPSIGTGELAYAVGSKLMDVRTMSKETHIAEEVSTSQQTSQNTAGSL
4 VEDSDYGPGGAGQSQKRESCSKSAELVGFNVHNDTSLYDSSNFVSSTEVNNSCKDSQDYNDMGEPYDTEFTDDYPSLPVTNFFSTDGVGS
5 NGVDKNHCSFVPEDLRLRHRDERMHSFQINNIDLIIESNCSDDVFRASNSDSAI FHSDAYKQDRWLDDNGYNREVIDSLRISDAPEADFV
6 DGTKNSVVASKDRVSEWLTLHRIIVDVVRTDSDLDFYGESRNMARMSDILAVYAWVDPSTGYCQMSDLLSPFVVLVEDDADAFWCFEML
7 LRRMRENFQMEGPTGVMKQLQALWKIMEITDVELFEHLSTIGAESLHFAFRMLLVLFRELSFEEISLWEMMMWAADFNEVDILHLEENCLE
8 PLLVDMRNDLSCEVKEEHRVNSYTRRKSRSRKP HHRNGEMRVACNLGMKPNTNRNPLCGLSGATIWARHQQMPHISTNVLAKNDDDLPIFCV
9 AAILVINRHKIIRETRSIDDAIKMFNDNMLKINVKRCVRMAIKLRKKYIYKLLKGGSE*
10
11 >LOC_Os01g01010.2
12 MSSAAGQDNGDTAGDYIKWMCAGGRAGGAMANLQRGVGSILVRDIDGDCPLNPSVKGSKMLKPEKWHTCFDNDGKIVIGFRKALKFIVLGGVD
13 PTIRAEVWEFLGCGYALSSTSEYRRKLRAVRREKYQILVRQCQSMHPSIGTGELAYAVGSKLMDVRTMSKETHIAEEVSTSQQTSQNTAGSL

```

Figure 3 The expected output from part 1b.

Assignment Part 2 (20%)

In this part, you should start a new python file, where we are going to use regular expressions to find all the [bzip](#) transcription factors (TFs) on rice chromosome 1. Using what you learnt in the week 5 lecture material, read in the proteins produced in part 1b, and check whether each one contains the characteristic “motif” of a bzip protein (Figure 4). In plain English, one could say that a bzip is characterised by A (position 442 of the alignment), any character, R or K, S or Y, R, any character and so on i.e. work out a motif from positions 442 to 449 on the alignment.

See if you can write and implement a regular expression to extract just the bzip proteins from the set produced in part 1b, and write a new fasta file containing these, with the motif position in the

⁴ For example, sourced from here <https://www.genscript.com/tools/codon-table>

header (Figure 5). If you didn't manage to complete that part correctly, you can always grab just the chromosome 1 proteins from the fasta file linked above. *We cannot give grades for code that does not use a regular expression.*

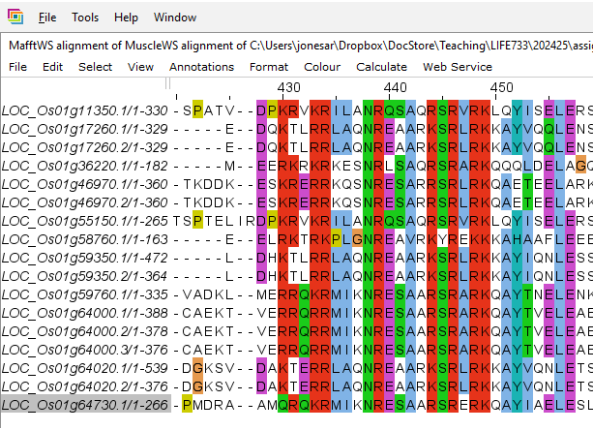


Figure 4 A portion of a protein multiple sequence alignment of bzip TF proteins on rice chromosome 1. The regions of amino acid shared between all sequences can be considered a characteristic motif of a bzip TF.

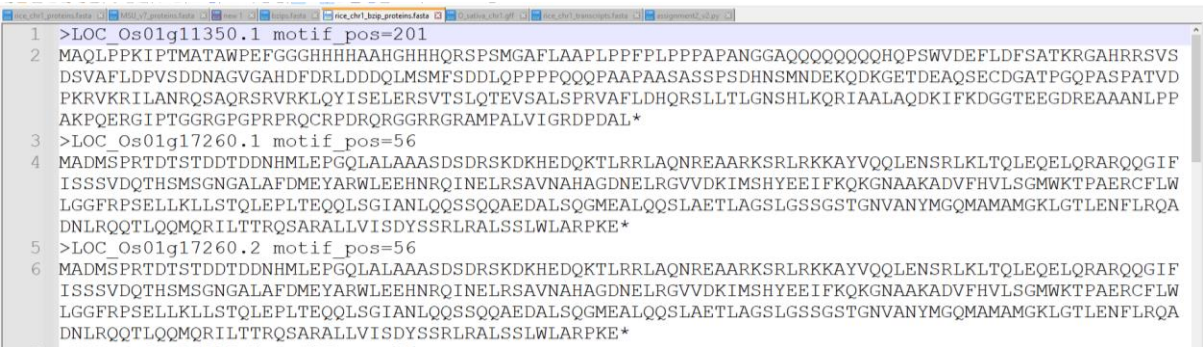


Figure 5 Expected output from part 2.

Rubric for marking Assignment 2

Note – start from Assignment 2 onwards, there are some points given for code style and quality i.e. some points will be deducted if you solve problems in a very convoluted or non-recommended way to help you improve your code going forward.

1a - FASTA reading function	1a - GFF reader and correct transcript output	1a - Code quality and documentation	1b - Translate to protein and write FASTA	1b - Code quality and documentation	2 - Find bZips using regular expression and correct fasta output	2 Code quality and documentation	Total
10	35	5	25	5	15	5	100